

Student name: Solomon Mwaura

Date: March 15, 2026

1. Problem Identification and Requirements

Objective: Clearly define the problem the IoT system is intended to solve. Identify the key requirements and constraints.

Response:

- **Describe the problem:** Precision Manufacturing Inc. faces significant operational inefficiencies, including unexpected equipment downtime, machinery overheating, and high energy costs due to machines idling.
 - **Data collection needs:** Real-time monitoring of machine **temperature** (to prevent overheating), **vibration** (to detect mechanical wear), **power consumption** (for energy audits), and **operational uptime**.
 - **Automated actions:** The system must trigger alerts when thresholds for heat or vibration are exceeded and use industrial relays to power down or throttle machines during idle periods to save energy.
 - **Constraints:** The system must operate in a high-interference industrial environment (EMI), maintain strict security to prevent industrial espionage, and ensure low-latency processing for safety-critical alerts.
-

2. Selection of Sensors and Edge Devices

Objective: Choose the appropriate sensors and actuators that will be used to collect data and perform automated actions.

Response:

- **List of sensors:**
 - * **Thermocouple Temperature Sensor:** Chosen for its durability and wide range in high-heat industrial environments.
 - **Accelerometer (Vibration Sensor):** Essential for identifying the harmonic signatures of failing bearings or misaligned shafts.
 - **Current Sensor:** Non-invasive clips to monitor real-time amperage and calculate power draw.
- **List of actuators:**
 - * **Industrial Relays:** These will act as the physical switch to automate energy-saving protocols by cutting or restoring power based on the production schedule.

- **Edge device selection:** * **Industrial Raspberry Pi / NVIDIA Jetson Nano:** Acts as the high-compute gateway.
 - **ESP32 Microcontroller:** Used at the machine level to aggregate sensor data and transmit it to the gateway via Wi-Fi or MQTT.
-

3. Connectivity and Communication Needs

Objective: Determine how the IoT system will communicate and transmit data between devices and the cloud.

Response:

- **Local or cloud-based system:** A hybrid approach. Local processing (Edge) is used for immediate alerts, while cloud storage is used for long-term predictive analytics.
 - **Communication protocols:** * **MQTT:** The primary lightweight "publish/subscribe" protocol for sensor-to-gateway communication.
 - **Wi-Fi:** Standard for factory-wide data backhaul.
 - **LoRaWAN:** Employed for long-range sensors in parts of the facility where Wi-Fi penetration is poor.
 - **Bandwidth and power considerations:** MQTT minimizes bandwidth usage. Using ESP32 allows for low-power sleep modes during non-operational hours to extend the life of battery-operated sensors.
-

4. Processing and Storage Options

Objective: Decide where data will be processed and stored based on system requirements.

Response:

- **Processing method:** **Edge Computing** (Raspberry Pi/Jetson Nano) handles real-time anomaly detection (e.g., stopping a machine if it hits 100°C). **Cloud Computing** (AWS IoT Greengrass) processes historical trends to predict *when* a machine might fail three months from now.
 - **Storage method:** Real-time logs are stored in a local **PostgreSQL** database. Aggregated, high-volume historical data is pushed to **AWS DynamoDB** for scalable cloud storage.
-

5. Security Measures Implementation

Objective: Ensure that the IoT system is secure against threats and unauthorized access.

Response:

- **Data protection methods:** **AES-256** encryption for data at rest on the edge device and **TLS/SSL** for all data in transit to the cloud.
 - **Access control strategy:** **Role-Based Access Control (RBAC)** ensures that only maintenance engineers can trigger relays, while management can only view the dashboard.
 - **System updates and maintenance:** Implementation of **Firmware Over-the-Air (FOTA)** updates to push security patches to all ESP32 and Gateway devices simultaneously without manual intervention.
-

6. User Interface Design

Objective: Define how users will interact with the IoT system and access real-time data.

Response:

- **Interface type:** A centralized **Web Dashboard** accessible via tablets on the factory floor and desktop computers in the engineering office.
 - **Dashboard features:** Real-time gauges for temperature/vibration, a "Red/Yellow/Green" status for machine health, energy savings reports, and a button for manual emergency override.
 - **User experience considerations:** The UI will use **React.js** for high responsiveness, featuring high-contrast visuals for readability in low-light factory environments.
-

7. Final System Architecture Summary

Objective: Summarize the complete IoT architecture and provide an overview of how the system components work together.

Response:

- **How all components integrate:** Sensors capture physical data → ESP32 transmits via MQTT → Industrial Gateway processes data locally for immediate action (Relays) → Gateway pushes data to AWS Cloud → Cloud analyzes trends and populates the React.js Dashboard.

- **Potential improvements:** Integrating AI/Machine Learning models directly on the Jetson Nano (TinyML) to recognize failure patterns locally without needing cloud intervention.
-

8. Conclusion

Objective: Provide a final reflection on the IoT system design process and the feasibility of the proposed solution.

Response:

- **Key takeaways:** The balance between Edge and Cloud is critical; real-time safety cannot depend on cloud latency.
- **Challenges faced:** Designing for industrial "noise" (EMI) and ensuring the network remains stable across a large metal-filled factory floor.
- **Next steps:** A pilot program on a single assembly line to calibrate the vibration "normal" baseline before scaling to the entire facility.